

AI Capstone HW3 Report 112550151 周佳瑩

1. Implementation (50%)

a. Code

Detailed explanation of your implementation with code screenshots.

For example:

- How do you process the 3D pointcloud?
- How do you do the RRT algorithm?

```
def splat_point_weights(
    rows_float: np.ndarray,
    cols_float: np.ndarray,
    img_height: int,
    img_width: int,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    """
    Rasterize points with bilinear weights onto the four nearest pixels.
    This fixes most visual holes at the projection stage instead of with
    extra post-processing passes.
    """
    row0 = np.floor(rows_float).astype(int)
    col0 = np.floor(cols_float).astype(int)
    drow = rows_float - row0
    dcol = cols_float - col0

    all_rows = []
    all_cols = []
    all_weights = []
    all_indices = []
    point_indices = np.arange(len(rows_float), dtype=int)

    for row_offset, row_weight in ((0, 1.0 - drow), (1, drow)):
        for col_offset, col_weight in ((0, 1.0 - dcol), (1, dcol)):
            rows = row0 + row_offset
            cols = col0 + col_offset
            weights = row_weight * col_weight

            valid = (
                (rows >= 0) & (rows < img_height) &
                (cols >= 0) & (cols < img_width) &
                (weights > 1e-6)
            )
            if np.any(valid):
                all_rows.append(rows[valid])
                all_cols.append(cols[valid])
                all_weights.append(weights[valid].astype(np.float32))
                all_indices.append(point_indices[valid])
```

```

if not all_rows:
    return (
        np.array([], dtype=int),
        np.array([], dtype=int),
        np.array([], dtype=np.float32),
        np.array([], dtype=int),
    )

    return (
        np.concatenate(all_rows),
        np.concatenate(all_cols),
        np.concatenate(all_weights),
        np.concatenate(all_indices),
    )

def remove_small_components(mask: np.ndarray, min_area: int) -> np.ndarray:
    """Drop tiny connected components caused by sparse or noisy point samples."""
    if min_area <= 1:
        return mask

    num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(mask, connectivity=8)
    filtered = np.zeros_like(mask)
    for label in range(1, num_labels):
        area = stats[label, cv2.CC_STAT_AREA]
        if area >= min_area:
            filtered[labels == label] = 1
    return filtered

def load_and_filter_map(point_path: str, color_path: str):

    points = np.load(point_path)
    colors = np.load(color_path)

    # Convert to real-world meters
    coords = points * SCALE_FACTOR

    # ===== TODO 1-1 =====
    # Hints: To get a good 2d map, filter ceiling/floor, project to 2D,
    # remove isolated points, inflate obstacles to get occupancy map, etc.
    # IMPORTANT: return map_img as float in value range [0, 1] for visualization
    downstream.

    # NOTE: in habitat sim, x z plane corresponds to world horizontal plane, and y is
    vertical.

    color_diff = colors.astype(int)
    ceiling_mask = np.all(np.abs(color_diff - CEILING_COLOR) == 0, axis=1)
    floor_mask = np.all(np.abs(color_diff - FLOOR_COLOR) == 0, axis=1)
    floor_height = float(np.median(coords[floor_mask, 1]))
    walkable_height_cutoff = floor_height + WALKABLE_SUPPORT_MAX_HEIGHT
    obstacle_min_height = floor_height + OBSTACLE_MIN_HEIGHT

```

```

obstacle_height_cutoff = floor_height + ROBOT_CLEARANCE_HEIGHT
walkable_support_mask = (~ceiling_mask) & (coords[:, 1] <= walkable_height_cutoff)
obstacle_mask = (
    (~ceiling_mask & ~floor_mask)
    & (coords[:, 1] >= obstacle_min_height)
    & (coords[:, 1] <= obstacle_height_cutoff)
)
# Rasterize into a top-down grid shared by the visualization and the planner.
# Hide low walkable-support surfaces such as carpet/rugs in both outputs.
display_candidate_mask = (~ceiling_mask & ~floor_mask) & (coords[:, 1] >=
walkable_height_cutoff)
display_candidate_coords = coords[display_candidate_mask]
display_candidate_colors = colors[display_candidate_mask]
floor_coords = coords[walkable_support_mask]
obstacle_coords = coords[obstacle_mask]

xs = display_candidate_coords[:, 0]
zs = display_candidate_coords[:, 2]
x_min, x_max = xs.min(), xs.max()
z_min, z_max = zs.min(), zs.max()

resolution = 0.05 # meters per pixel
img_width = int((x_max - x_min) / resolution) + 1
img_height = int((z_max - z_min) / resolution) + 1

def project_to_grid(points_3d: np.ndarray):
    if len(points_3d) == 0:
        return (
            np.array([], dtype=np.float32),
            np.array([], dtype=np.float32),
        )
    cols = (points_3d[:, 0] - x_min) / resolution
    rows = (img_height - 1) - ((points_3d[:, 2] - z_min) / resolution)
    return rows.astype(np.float32), cols.astype(np.float32)

display_rows, display_cols = project_to_grid(display_candidate_coords)
floor_rows, floor_cols = project_to_grid(floor_coords)
obstacle_rows, obstacle_cols = project_to_grid(obstacle_coords)

# Build a walkable-space map:
# 0 = free space, 1 = occupied or unknown.
floor_density = np.zeros((img_height, img_width), dtype=np.float32)
floor_splat_rows, floor_splat_cols, floor_weights, _ = splat_point_weights(
    floor_rows, floor_cols, img_height, img_width
)
if floor_weights.size > 0:
    np.add.at(floor_density, (floor_splat_rows, floor_splat_cols), floor_weights)

obstacle_density = np.zeros((img_height, img_width), dtype=np.float32)
obstacle_splat_rows, obstacle_splat_cols, obstacle_weights, _ = splat_point_weights(
    obstacle_rows, obstacle_cols, img_height, img_width
)

```

```

if obstacle_weights.size > 0:
    np.add.at(obstacle_density, (obstacle_splat_rows, obstacle_splat_cols),
obstacle_weights)

floor_mask_2d = (floor_density > 0.10).astype(np.uint8)
obstacle_mask_2d = (obstacle_density > 0.08).astype(np.uint8)

floor_mask_2d = remove_small_components(floor_mask_2d, MIN_FLOOR_COMPONENT_PIXELS)
obstacle_mask_2d = remove_small_components(obstacle_mask_2d,
MIN_OBSACLE_COMPONENT_PIXELS)

# Grow walkable support from all low-height surfaces so rugs and doorway
# thresholds can reopen even when they are not labeled with the exact
# floor semantic color.
floor_kernel = np.ones((5, 5), np.uint8)
obstacle_kernel = np.ones((3, 3), np.uint8)
floor_region = cv2.morphologyEx(floor_mask_2d, cv2.MORPH_CLOSE, floor_kernel)
floor_region = cv2.dilate(floor_region, floor_kernel, iterations=1)
obstacle_region = cv2.morphologyEx(obstacle_mask_2d, cv2.MORPH_CLOSE,
obstacle_kernel)
obstacle_region = cv2.dilate(obstacle_region, obstacle_kernel, iterations=1)

occupancy_map = np.ones((img_height, img_width), dtype=np.uint8)
occupancy_map[floor_region > 0] = 0
occupancy_map[obstacle_region > 0] = 1

# Inflate obstacles for safer planning while keeping only floor-supported space
free.
if ROBOT_RADIUS_METER > 0:
    inflate_pixels = max(1, int(round(ROBOT_RADIUS_METER / resolution)))
    inflate_kernel = np.ones((2 * inflate_pixels + 1, 2 * inflate_pixels + 1),
np.uint8)
    inflated_obstacles = cv2.dilate(obstacle_region, inflate_kernel, iterations=1)
    inflated_occupancy = occupancy_map.copy()
    inflated_occupancy[inflated_obstacles > 0] = 1
else:
    inflated_occupancy = occupancy_map

# Apply the same final obstacle decisions back to the original points so
# filtered obstacle points do not remain blocked only visually.
display_point_obstacle_flag = obstacle_mask[display_candidate_mask]
display_rows_int = np.clip(np.floor(display_rows).astype(int), 0, img_height - 1)
display_cols_int = np.clip(np.floor(display_cols).astype(int), 0, img_width - 1)
kept_obstacle_points = inflated_occupancy[display_rows_int, display_cols_int] != 0
display_keep_mask = (~display_point_obstacle_flag) | kept_obstacle_points

display_coords = display_candidate_coords[display_keep_mask]
display_colors = display_candidate_colors[display_keep_mask]
xs = display_coords[:, 0]
zs = display_coords[:, 2]
colors_norm = display_colors / 255.0
display_rows = display_rows[display_keep_mask]

```

```

display_cols = display_cols[display_keep_mask]

raster_map = np.ones((img_height, img_width, 3), dtype=np.float32)
color_sum = np.zeros((img_height, img_width, 3), dtype=np.float32)
color_weight = np.zeros((img_height, img_width), dtype=np.float32)

raster_rows, raster_cols, raster_weights, raster_indices = splat_point_weights(
    display_rows, display_cols, img_height, img_width
)
if raster_weights.size > 0:
    np.add.at(color_sum, (raster_rows, raster_cols), colors_norm[raster_indices] *
raster_weights[:, None])
    np.add.at(color_weight, (raster_rows, raster_cols), raster_weights)

colored_pixels = color_weight > DISPLAY_COLOR_WEIGHT_THRESHOLD
raster_map[colored_pixels] = (
    color_sum[colored_pixels] / color_weight[colored_pixels, None]
)

# Scatter plot rendered to numpy array for select window
fig, ax = plt.subplots(figsize=(10, 8))
ax.scatter(xs, zs, c=colors_norm, s=1)
ax.set_title("2D Semantic Map")
ax.set_xlabel("X (world units)")
ax.set_ylabel("Z (world units)")
ax.axis("equal")
plt.tight_layout()

# Render figure to a numpy array (RGB)
fig.canvas.draw()
x_lim = ax.get_xlim()
z_lim = ax.get_ylim()
fig_w, fig_h = fig.canvas.get_width_height()

buf = np.asarray(fig.canvas.buffer_rgba())[..., :3]
map_img = buf.astype(np.float32) / 255.0
plt.close(fig)

return (raster_map, occupancy_map, x_min, z_min, resolution, img_height)

```

I do map construction by first loading the point cloud and color arrays and converting the coordinates to real-world meters using the scale factor. Then I implement the filtering logic. I separate the points into ceiling, floor, walkable-support, and obstacle categories according to their height above the detected floor level and their semantic color, do a projection to convert the 3D points top-down onto a 2D grid at 5 cm/pixel resolution, and use bilinear splatting to distribute each point's contribution across its four neighboring pixels to avoid holes in the raster. Next, I threshold the density maps into binary floor

and obstacle masks, apply connected component filtering to remove small noisy clusters, and use morphological closing and dilation to fill gaps and smooth edges. I then build the occupancy map by marking floor regions as free and obstacle regions as occupied, and inflate the obstacles by the robot radius to ensure safe clearance. The start position is selected by the user clicking on a CV2 window displaying the raster map.

```
def RRT(start, goal, occupancy_map, stepSize=10, max_iter=5000, goal_sample_rate=0.15):
    global LAST_RRT_EDGES, LAST_RRT_NODES
    h, w = occupancy_map.shape
    node_list: List[Nodes] = [Nodes(start[0], start[1])]
    LAST_RRT_EDGES = []
    LAST_RRT_NODES = [start]

    for _ in range(max_iter):
        if random.random() < goal_sample_rate:
            nx, ny = goal
        else:
            nx, ny = rnd_point(h, w)

        nearest_ind = nearest_node(nx, ny, node_list)
        nearest_x = node_list[nearest_ind].x
        nearest_y = node_list[nearest_ind].y

        tx, ty, directCon, nodeCon = check_collision(
            nx, ny, nearest_x, nearest_y, goal, occupancy_map, stepSize
        )

        if not nodeCon:
            continue

        if (tx, ty) == (nearest_x, nearest_y):
            continue

        if any(node.x == tx and node.y == ty for node in node_list):
            continue

        new_node = Nodes(tx, ty, parent=node_list[nearest_ind])
        node_list.append(new_node)
        LAST_RRT_EDGES.append(((nearest_x, nearest_y), (tx, ty)))
        LAST_RRT_NODES.append((tx, ty))

        if directCon:
            LAST_RRT_EDGES.append(((tx, ty), goal))
            path = densify_path(backtrack_path(new_node, goal))
            if path_is_collision_free(path, occupancy_map):
                return path

    return None
```

```

def plan_path(start, goal, occupancy_map):
    global LAST_RRT_EDGES, LAST_RRT_NODES
    start = (int(start[0]), int(start[1]))
    goal = (int(goal[0]), int(goal[1]))
    LAST_RRT_EDGES = []
    LAST_RRT_NODES = [start]

    start_free = nearest_free_point(start, occupancy_map)
    goal_free = nearest_free_point(goal, occupancy_map)

    if start_free is None or goal_free is None:
        return None

    if not collision(start_free[0], start_free[1], goal_free[0], goal_free[1],
occupancy_map):
        return densify_path([start_free, goal_free])

    step_size = max(6, min(15, min(occupancy_map.shape) // 40))
    for _ in range(3):
        path = RRT(start_free, goal_free, occupancy_map, stepSize=step_size,
max_iter=8000)
        if path and path[-1] != goal_free and not collision(
            path[-1][0], path[-1][1], goal_free[0], goal_free[1], occupancy_map
        ):
            path = densify_path(path + [goal_free])
            if path_is_collision_free(path, occupancy_map) and path[-1] == goal_free:
                return path
    return None

```

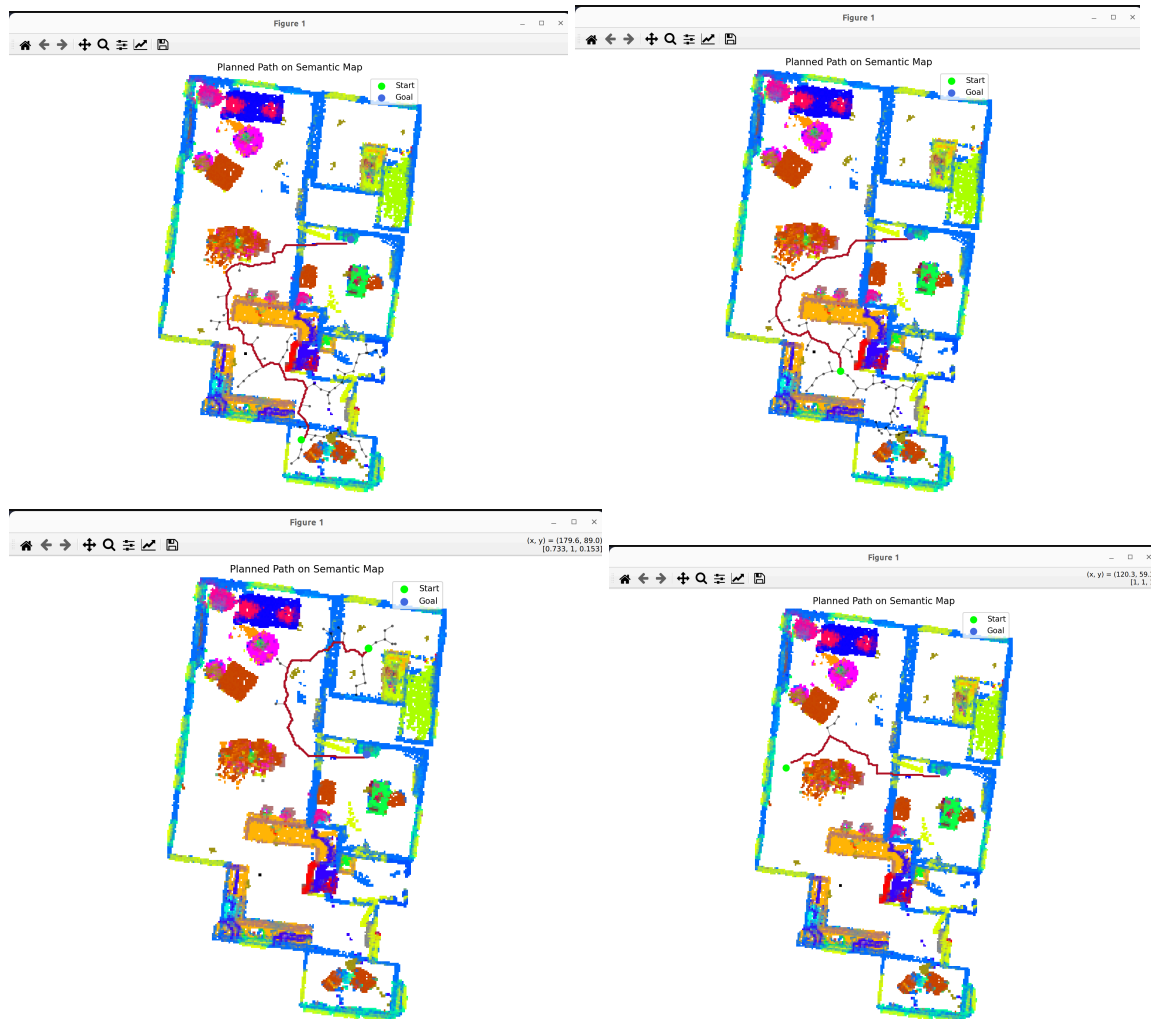
In the path planner, I first snap the start and goal to the nearest free pixels if they fall inside obstacles. For global connectivity, I run a BFS from the start to find all reachable free space, and use that to validate that the goal is reachable. I implement RRT by repeatedly sampling random pixels or the goal directly, finding the nearest existing tree node, stepping toward the sample by one step size, and checking the new segment for collisions by stepping along it at 1-pixel increments. When a new node has a direct line of sight to the goal, I backtrack through parent pointers to reconstruct the path. The final path is densified by interpolating intermediate pixels between waypoints.

Lastly, I implement world-space translation and simulator execution by converting each raster pixel to Habitat world coordinates accounting for the grid origin, resolution, and flipped row axis, and compress the dense path into coarser waypoints separated by at least one move step. For each segment I compute the required heading change, issue the appropriate number of turn actions to align the agent, then step forward to cover the distance, tracking the

current heading throughout.

b. Result and Discussion

- Show and discuss the results from the RRT algorithm with different start points and targets.
- Discuss about the robot navigation results.
- Anything you want to discuss.



Due to the randomness of the algorithm, even when there's a clear path ahead, the robot doesn't take that clear path, and instead might make extra turns to get to the desired point.

c. Reference

- <https://github.com/nimRobotics/RRT>
- <https://medium.com/@aggorjefferson/exploring-path-planning-with-rrt-and-visualization-in-python-cf5bd80a6cd6>

2. Questions (20%)

- In the RRT algorithm, you can adjust the step size and bias (the

number balance between exploration and exploitation). Please explain how the two numbers affect the RRT sampling result. (15%)

A larger step size makes the tree grow faster and cover space more quickly, but it is more likely to skip over thin obstacles since the collision checks only samples along the segment. On the other hand, a large step can leap over an obstacle and register as collision-free. A smaller step size is more precise around tight spaces but requires more iterations to reach the goal.

A higher bias makes the tree grow more directly toward the goal, which speeds up planning in open spaces but can get stuck if obstacles block the direct route, since it keeps trying to extend toward the goal instead of exploring around it. A lower bias favors random exploration, which handles complex obstacles better but wastes iterations expanding in irrelevant directions.

- b. If you want to apply the indoor navigation pipeline in the real world, what problems may you encounter? (5%)

Several problems arise in the real world. The pipeline assumes a pre-built perfectly labeled point cloud, but in reality the map must be built on the fly with noisy sensors. Semantic color matching would not work under real lighting conditions and would need to be replaced with a proper perception model. The occupancy map also assumes a static environment, but real spaces have moving people and changing furniture that would make the map go stale. Finally, real robots have wheel slip and motor imprecision that cause the trajectory to deviate from the plan, requiring feedback control to correct.